SHETH L.U.J AND SIR M.V COLLEGE
PRACTICAL NO 2
SUBJECT - AI

AIM - Implement Iterative deep depth first search for Romanian map problem.
CODE -

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = {

    "Andheri East": {
        "NH48": 10
    },

    #  Route 1 (155 km)
    "NH48": {
        "Mumbai Nashik Expressway": 127,
        "NH160": 127,
        "Andad": 66.5
    },

    "Mumbai Nashik Expressway": {
        "Khardi-Wada Road": 18
    },

    "Khardi-Wada Road": {
        "Shivneri Fort": 0
    },

    #  Route 2 (155 km)
    "NH160": {
        "Khardi-Wada Road 2": 18
    },

    "Khardi-Wada Road 2": {
        "Shivneri Fort": 0
    },

    #  Route 3 (179 km)
    "Andad": {
        "Saralgaon": 30
    },

    "Saralgaon": {
        "NH61": 54.5
    },

    "NH61": {
        "Shivneri Fort": 18
    },

    "Shivneri Fort": {}
}
```

NAME - UNNATI RATHOD
ROLL NO - T109

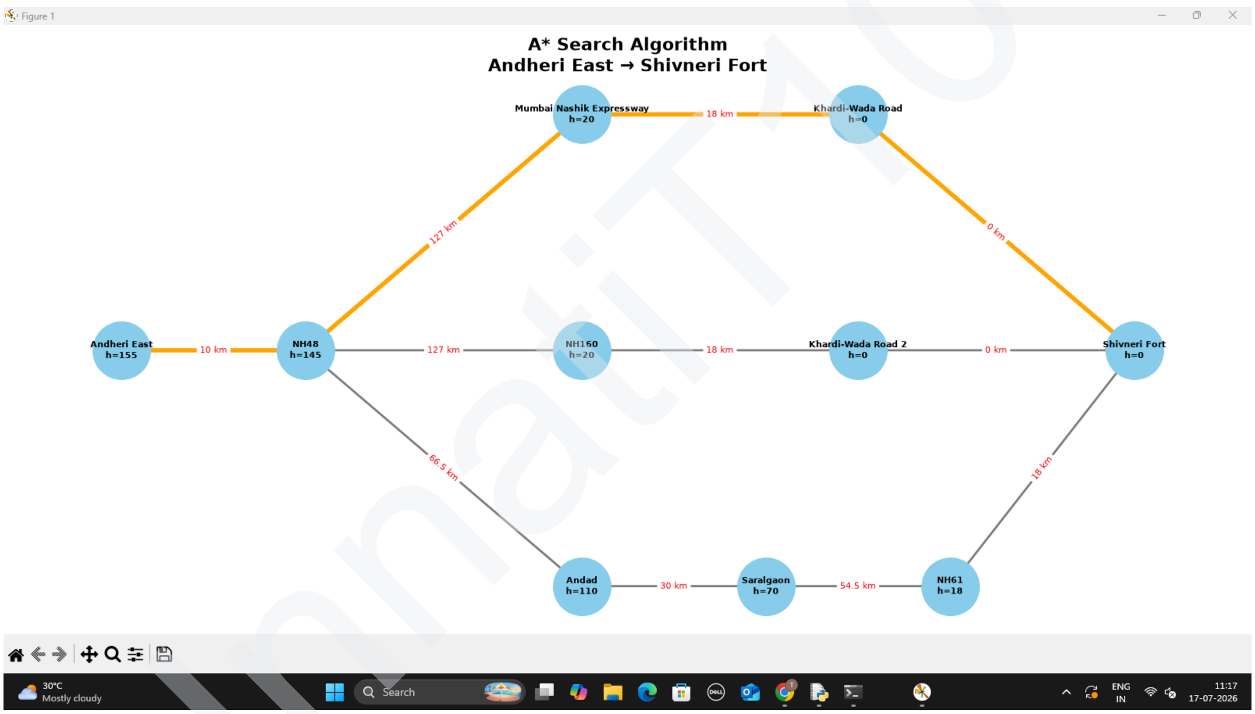
```
heuristics = {

    "Andheri East": 155,

    "NH48": 145,

    "Mumbai Nashik Expressway": 20,
    "Khardi-Wada Road": 0,

    "NH160": 20,
    "Khardi-Wada Road 2": 0,

    "Andad": 110,
    "Saralgaon": 70,
    "NH61": 18,

    "Shivneri Fort": 0
}

node_positions = {

    "Andheri East": (0,5),

    "NH48": (2,5),

    # Route 1
    "Mumbai Nashik Expressway": (5,7),
    "Khardi-Wada Road": (8,7),

    # Route 2
    "NH160": (5,5),
    "Khardi-Wada Road 2": (8,5),

    # Route 3
    "Andad": (5,3),
    "Saralgaon": (7,3),
    "NH61": (9,3),

    "Shivneri Fort": (11,5)
}


def a_star_search(graph, heuristics, start, goal):

    priority_queue = [(heuristics[start], start, [start], 0)]

    visited = set()

    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)
```

```
        if current in visited:
            continue

        visited.add(current)

        if current == goal:
            return path, g_score

        for neighbor, distance in graph[current].items():

            if neighbor not in visited:

                new_g = g_score + distance
                new_f = new_g + heuristics[neighbor]

                heapq.heappush(
                    priority_queue,
                    (new_f, neighbor, path + [neighbor], new_g)
                )

    return None, float("inf")

optimal_path, total_distance = a_star_search(
    graph,
    heuristics,
    "Andheri East",
    "Shivneri Fort"
)

print("="*50)
print("A* SEARCH RESULT")
print("="*50)

print("\nOptimal Path:")
print(" -> ".join(optimal_path))

print("\nTotal Distance:", total_distance, "km")

G = nx.DiGraph()

for node, neighbours in graph.items():
    for neighbour, weight in neighbours.items():
        G.add_edge(node, neighbour, weight=weight)

plt.figure(figsize=(16,8))

path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [
    edge for edge in G.edges()
    if edge not in path_edges
]
```

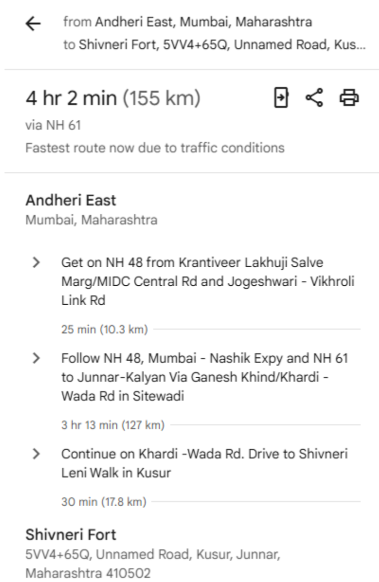
```
nx.draw_networkx_nodes(
    G,
    node_positions,
    node_color="skyblue",
    node_size=2800
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    edge_color="gray",
    width=2,
    arrows=True,
    arrowsize=20
)

nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    edge_color="orange",
    width=4,
    arrows=True,
    arrowsize=20
)

labels = {
    node: f"{node}\nh={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels,
    font_size=8,
    font_weight="bold"
)

edge_labels = nx.get_edge_attributes(G, "weight")

edge_labels = {
    edge: f"{weight} km"
    for edge, weight in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
```

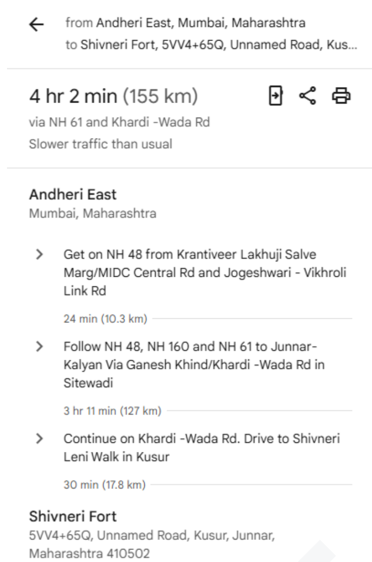
```
    font_color="red",
    font_size=8
)

plt.title(
    "A* Search Algorithm\nAndheri East → Shivneri Fort",
    fontsize=16,
    fontweight="bold"
)

plt.axis("off")
plt.tight_layout()
plt.show()
```

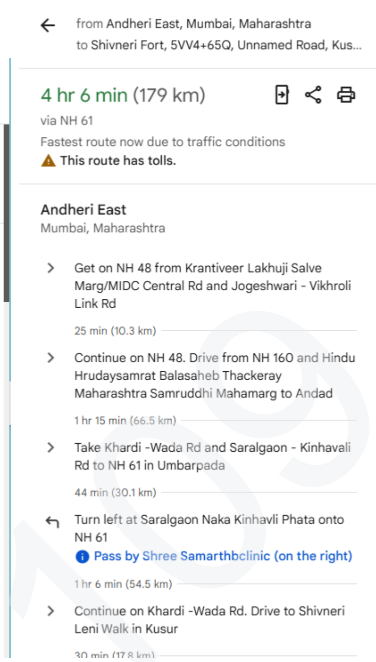
OUTPUT



SHETH L.U.J AND SIR M.V COLLEGE
PRACTICAL NO 2
SUBJECT - AI







NAME - UNNATI RATHOD
ROLL NO - T109